يهدف هذا المشروع إلى تطوير نظام إلكتروني لإدارة درجات الطلاب باستخدام تقنيات **Node.js** و **Express.js** مع واجهات أمامية مبنية باستخدام **HTML** و**CSS** و**JavaScript**. بدأ المشروع في نسخته الأولى كنظام بسيط يتيح إدخال بيانات الطالب فقط، ثم جرى تطويره تدريجيًا أثناء دراسة مادة بناء البرمجيات حتى أصبح نظامًا أكثر تنظيمًا واحترافية.

اعتمدت عملية التطوير على تطبيق عدد كبير من مفاهيم بناء البرمجيات، مثل **التجريد (Abstraction)**، **التغليف (Encapsulation)**، **الوحدات النمطية (Modularity)**، **إخفاء المعلومات (Information Hiding)**، بالإضافة إلى تطبيق مبادئ **الكود النظيف (Clean Code)** وتقليل تكرار الأكواد باستخدام مبدأ **DRY**.

كما تم تحسين تصميم المشروع من خلال فصل مسؤوليات النظام إلى ملفات مستقلة، مما أدى إلى تحسين قابلية الصيانة والتطوير وإعادة الاستخدام، وأصبح المشروع أكثر مرونة في إضافة ميزات جديدة دون التأثير على الأجزاء الأخرى من النظام.

يعرض هذا التقرير جميع مراحل تطوير المشروع، ويبين كيفية تطبيق مفاهيم مادة بناء البرمجيات عمليًا على مشروع حقيقي، مع تقديم أمثلة من الأكواد البرمجية قبل وبعد التحسين، وشرح أثر كل تحسين على جودة النظام.

الفصل الأول: المقدمة (Introduction)

أصبحت الأنظمة الإلكترونية جزءًا أساسيًا في المؤسسات التعليمية الحديثة، حيث تعتمد الجامعات والكليات على برامج متخصصة لإدارة بيانات الطلاب ودرجاتهم، وذلك بهدف تنظيم المعلومات وتقليل الأخطاء الناتجة عن العمل اليدوي.

ومن هذا المنطلق، تم تطوير مشروع **نظام إدارة درجات الطلاب** ليكون نموذجًا عمليًا لتطبيق مفاهيم مادة **بناء البرمجيات**، بحيث لا يقتصر المشروع على تنفيذ الوظائف الأساسية فقط، بل يركز أيضًا على جودة بناء النظام، وتنظيم الكود، وسهولة تطويره مستقبلًا.

في البداية، كان المشروع عبارة عن صفحة بسيطة تسمح للمستخدم بإدخال بيانات الطالب فقط، وكانت جميع العمليات البرمجية موجودة داخل ملف واحد، مما أدى إلى صعوبة قراءة الكود وصيانته وإضافة ميزات جديدة.

ومع دراسة موضوعات المادة، تم إعادة تصميم المشروع تدريجيًا وفقًا لمبادئ هندسة البرمجيات الحديثة، حيث تم فصل الوظائف إلى وحدات مستقلة، وتحسين أسماء المتغيرات والدوال، وإعادة تنظيم الملفات، وتطبيق أنماط التصميم المناسبة، مما ساهم في إنتاج نظام أكثر احترافية.

ولا يهدف هذا التقرير إلى وصف المشروع فقط، بل يوضح أيضًا كيف تم تحويل المفاهيم النظرية التي تمت دراستها في المحاضرات إلى تطبيقات عملية داخل المشروع، بحيث يمكن ملاحظة أثر كل مفهوم على جودة البرمجية وأدائها.

الفصل الثاني: مشكلة المشروع (Problem Statement)

عند البدء في تنفيذ المشروع، كان الهدف الأساسي هو إنشاء نظام بسيط يسمح بإضافة بيانات الطالب وحفظها داخل النظام، إلا أن النسخة الأولية كانت محدودة الإمكانيات، ولم تكن مناسبة للاستخدام الفعلي داخل مؤسسة تعليمية.

ومن خلال تحليل النسخة الأولى، تم تحديد مجموعة من المشكلات الرئيسية.

أولاً: غياب خاصية البحث

كان النظام يسمح بإضافة بيانات الطالب فقط، ولكن لم يكن بالإمكان البحث عن طالب معين باستخدام الاسم أو رقم القيد.

وهذا يعني أنه عند زيادة عدد الطلاب يصبح الوصول إلى بيانات طالب معين أمرًا صعبًا للغاية.

ثانياً: عدم إمكانية تعديل البيانات

بعد إدخال بيانات الطالب، لا يستطيع المستخدم تعديل الاسم أو الدرجة أو رقم القيد.

وفي حال وجود خطأ في البيانات، كان الحل الوحيد هو إعادة إدخال السجل بالكامل، مما يؤدي إلى ضياع الوقت وزيادة احتمال حدوث أخطاء جديدة.

ثالثاً: عدم وجود صفحة رئيسية

كانت جميع العمليات تتم داخل صفحة واحدة فقط، لذلك كان استخدام النظام غير منظم، ولا توجد طريقة واضحة للتنقل بين الوظائف المختلفة.

رابعاً: ضعف تصميم واجهة المستخدم

لم يكن التصميم يراعي تجربة المستخدم، حيث كانت الواجهة بسيطة جدًا، ولا تحتوي على تنظيم جيد للأزرار أو الألوان أو أماكن عرض البيانات، مما يقلل من سهولة استخدام النظام.

خامساً: تكرار الأكواد البرمجية

احتوى المشروع في بدايته على العديد من الأكواد المتكررة، حيث كانت بعض الوظائف تُكتب أكثر من مرة داخل المشروع، مما يزيد من حجم الكود ويجعل صيانته أكثر صعوبة.

سادساً: عدم تنظيم ملفات المشروع

كانت معظم الوظائف موجودة داخل ملف واحد، مما أدى إلى زيادة حجم الملف وصعوبة فهمه، كما أن أي تعديل صغير قد يؤثر على بقية أجزاء النظام.

سابعاً: ضعف قابلية التوسع

لم يكن بالإمكان إضافة ميزات جديدة بسهولة، مثل:

- إنشاء تقارير.
- حذف طالب.
- تصدير البيانات.
- إضافة مستخدمين.
- إنشاء صلاحيات مختلفة.

وذلك لأن تصميم المشروع لم يكن يعتمد على بنية منظمة.

الفصل الثالث: أهداف المشروع (Project Objectives)

يهدف المشروع إلى تحقيق مجموعة من الأهداف التعليمية والعملية التي تتوافق مع مفاهيم مادة بناء البرمجيات، ومن أهم هذه الأهداف:

1. تطوير نظام لإدارة درجات الطلاب

إنشاء نظام إلكتروني يسمح بإضافة بيانات الطلاب وإدارتها بطريقة سهلة ومنظمة.

2. تحسين جودة الكود البرمجي

تطبيق مبادئ Clean Code من خلال كتابة كود واضح، منظم، وسهل القراءة.

3. تطبيق مبادئ بناء البرمجيات

مثل:

- Abstraction
- Encapsulation
- Modularity
- Information Hiding

4. تنظيم ملفات المشروع

تقسيم المشروع إلى ملفات مستقلة بحيث تكون لكل وحدة مسؤولية محددة، مما يسهل تطوير النظام وصيانته.

5. تحسين تجربة المستخدم

تصميم واجهات سهلة الاستخدام، مع توفير تنقل واضح بين صفحات النظام.

6. تسهيل عمليات إدارة الطلاب

يشمل ذلك:

- إضافة طالب.
- تعديل بياناته.
- البحث عنه.
- عرض جميع الطلاب.

7. تقليل الأخطاء البرمجية

من خلال التحقق من صحة البيانات قبل تخزينها، ومعالجة الأخطاء بطريقة صحيحة.

8. تسهيل التطوير المستقبلي

تصميم المشروع بطريقة تسمح بإضافة ميزات جديدة دون الحاجة إلى إعادة كتابة أجزاء كبيرة من الكود.

الفصل الرابع: أهمية المشروع (Project Importance)

يعد مشروع **نظام إدارة درجات الطلاب** من المشاريع التعليمية المهمة التي تجمع بين الجانب النظري والجانب العملي في مجال تطوير البرمجيات، حيث يساهم في تطبيق مفاهيم بناء البرمجيات بصورة واقعية على نظام يمكن استخدامه داخل المؤسسات التعليمية.

تكمن أهمية المشروع في أنه يساعد على أتمتة عملية إدارة بيانات الطلاب، مما يقلل من الاعتماد على السجلات الورقية ويحد من الأخطاء البشرية الناتجة عن الإدخال اليدوي. كما يتيح للمستخدم إمكانية الوصول إلى بيانات الطلاب بسرعة، والبحث عنها وتعديلها عند الحاجة، الأمر الذي يرفع من كفاءة العمل ويوفر الوقت والجهد.

ومن الناحية التعليمية، يمثل المشروع بيئة مناسبة لتطبيق المفاهيم التي تمت دراستها في مادة بناء البرمجيات، مثل كتابة الكود النظيف، وتقسيم النظام إلى وحدات مستقلة، وتحسين جودة التصميم البرمجي، واستخدام أنماط التصميم المناسبة. ويساعد ذلك في اكتساب خبرة عملية في كيفية تطوير الأنظمة البرمجية وفقًا للمعايير الحديثة.

كما أن المشروع يوفر أساسًا يمكن البناء عليه مستقبلًا، حيث يمكن إضافة خصائص جديدة مثل إدارة المستخدمين، وإدارة المواد الدراسية، وإصدار التقارير، وربط النظام بقاعدة بيانات متقدمة دون الحاجة إلى إعادة تصميم النظام بالكامل، وذلك بفضل الهيكلية المنظمة التي تم اعتمادها أثناء التطوير.

وباختصار، فإن أهمية المشروع لا تقتصر على كونه نظامًا لإدارة درجات الطلاب، بل تمتد إلى كونه نموذجًا تطبيقيًا يوضح كيفية تحويل المفاهيم النظرية في بناء البرمجيات إلى نظام عملي قابل للتطوير والصيانة.

الفصل الخامس: نطاق المشروع (Project Scope)

يحدد نطاق المشروع الوظائف التي يقدمها النظام، بالإضافة إلى الحدود التي لا يشملها الإصدار الحالي، وذلك لتوضيح ما يستطيع النظام تنفيذه وما سيتم تركه للإصدارات المستقبلية.

أولًا: الوظائف التي يشملها المشروع

يشمل النظام مجموعة من الوظائف الأساسية التي يحتاجها المستخدم لإدارة بيانات الطلاب، وهي:

1. إضافة طالب جديد

يمكن للمستخدم إدخال البيانات الأساسية الخاصة بالطالب، والتي تشمل:

- اسم الطالب.
- رقم القيد.
- الدرجة.

بعد التحقق من صحة البيانات، يتم حفظها داخل النظام لتصبح متاحة للاستخدام في العمليات الأخرى.

---

2. البحث عن طالب

يوفر النظام إمكانية البحث عن طالب باستخدام أكثر من معيار، مثل:

- رقم القيد.
- اسم الطالب.

وتساعد هذه الخاصية في الوصول السريع إلى بيانات الطالب دون الحاجة إلى استعراض جميع السجلات.

---

3. تعديل بيانات الطالب

يمكن للمستخدم تحديث بيانات الطالب بعد إضافتها، سواء كان التعديل متعلقًا بالاسم أو رقم القيد أو الدرجة.

وتضمن هذه الخاصية بقاء البيانات محدثة وصحيحة.

---

4. عرض جميع الطلاب

يعرض النظام قائمة بجميع الطلاب المسجلين داخل جدول منظم يحتوي على أهم بيانات كل طالب، مما يسهل عملية المراجعة وإدارة السجلات.

---

5. تحسين واجهة المستخدم

تم تصميم واجهات النظام بحيث تكون سهلة الاستخدام، مع توزيع مناسب للعناصر والأزرار والقوائم، مما يسهل على المستخدم تنفيذ العمليات المختلفة.

ثانيًا: ما لا يشمله المشروع

حتى يبقى المشروع ضمن الحدود المطلوبة للمقرر الدراسي، فقد تم استبعاد بعض الخصائص المتقدمة، ومنها:

- نظام تسجيل الدخول.
- إدارة صلاحيات المستخدمين.

- إدارة أعضاء هيئة التدريس.
- إدارة المقررات الدراسية.
- حساب المعدل التراكمي.
- استخراج كشوف الدرجات الرسمية.
- إرسال الإشعارات عبر البريد الإلكتروني.
- النسخ الاحتياطي التلقائي للبيانات.

ويمكن إضافة هذه الوظائف مستقبلًا دون الحاجة إلى إعادة بناء النظام بالكامل.

الفصل السادس: تحليل المتطلبات (Requirements Analysis)

يعد تحليل المتطلبات من أهم مراحل تطوير البرمجيات، لأنه يحدد بدقة ما يجب أن يقدمه النظام للمستخدم، بالإضافة إلى الخصائص التي يجب أن يتمتع بها لضمان جودة الأداء.

وتنقسم المتطلبات إلى قسمين رئيسيين:

- المتطلبات الوظيفية (Functional Requirements).
- المتطلبات غير الوظيفية (Non-Functional Requirements).

أولًا: المتطلبات الوظيفية (Functional Requirements)

المتطلبات الوظيفية هي الخدمات والعمليات التي يجب أن يكون النظام قادرًا على تنفيذها.

1. إضافة بيانات طالب

يجب أن يسمح النظام للمستخدم بإضافة طالب جديد من خلال إدخال:

- الاسم.
- رقم القيد.
- الدرجة.

ويجب التحقق من صحة البيانات قبل حفظها داخل النظام.

---

2. البحث عن طالب

يجب أن يتمكن المستخدم من البحث عن أي طالب باستخدام الاسم أو رقم القيد، مع عرض النتائج بسرعة ودقة.

---

3. تعديل بيانات الطالب

بعد العثور على الطالب، يجب أن يسمح النظام بتعديل بياناته ثم حفظ التعديلات مباشرة.

---

4. عرض بيانات الطلاب

يجب أن يعرض النظام جميع الطلاب داخل جدول مرتب يوضح:

- اسم الطالب.
- رقم القيد.
- الدرجة.

---

5. التنقل بين الصفحات

يجب أن يستطيع المستخدم الانتقال بسهولة بين صفحات النظام المختلفة مثل:

- الصفحة الرئيسية.
- صفحة إضافة طالب.
- صفحة البحث.
- صفحة تعديل البيانات.
- صفحة عرض الطلاب.
- صفحة التقارير.

---

6. التحقق من صحة البيانات

قبل تنفيذ أي عملية، يجب أن يتحقق النظام من:

- عدم ترك الحقول فارغة.
- صحة رقم القيد.
- إدخال الدرجة ضمن النطاق المسموح.
- منع إدخال بيانات غير صحيحة.

ثانيًا: المتطلبات غير الوظيفية (Non-Functional Requirements)

تركز المتطلبات غير الوظيفية على جودة النظام وأدائه، وليس فقط على الوظائف التي يقدمها.

1. سهولة الاستخدام

يجب أن تكون واجهة المستخدم بسيطة وواضحة بحيث يستطيع أي مستخدم التعامل معها دون الحاجة إلى تدريب مسبق.

---

2. الأداء

ينبغي أن يستجيب النظام بسرعة عند تنفيذ عمليات الإضافة أو البحث أو التعديل، حتى مع زيادة عدد السجلات.

---

3. الاعتمادية

يجب أن يعمل النظام بصورة مستقرة دون توقف مفاجئ، مع ضمان عدم فقدان البيانات أثناء تنفيذ العمليات المختلفة.

---

4. قابلية الصيانة

تم تنظيم المشروع بطريقة تجعل من السهل تعديل أي جزء من الكود أو إضافة ميزات جديدة دون التأثير على بقية أجزاء النظام.

---

5. قابلية التوسع

يسمح تصميم المشروع بإضافة وظائف مستقبلية مثل:

- حذف الطلاب.
- إدارة المواد الدراسية.
- حساب المعدلات.

- ربط النظام بقاعدة بيانات SQL أو MongoDB.
- إنشاء حسابات للمستخدمين.

---

6. الأمان

على الرغم من أن الإصدار الحالي لا يحتوي على نظام تسجيل دخول، فقد تم تصميم المشروع بحيث يمكن مستقبلاً إضافة طبقات حماية مثل التحقق من الهوية والصلاحيات دون إجراء تغييرات جذرية في بنية النظام.

الفصل السابع: تحليل النظام (System Analysis)

مقدمة

تُعد مرحلة تحليل النظام (System Analysis) من أهم مراحل دورة حياة تطوير البرمجيات (Software Development Life Cycle - SDLC)، حيث يتم خلالها دراسة النظام بصورة دقيقة لفهم كيفية عمله، وتحديد مكوناته، وتحليل احتياجات المستخدمين، والعلاقات بين أجزاء النظام المختلفة.

في هذا المشروع، يهدف تحليل النظام إلى دراسة آلية إدارة بيانات الطلاب ودرجاتهم، وتحديد كيفية انتقال البيانات بين المستخدم والخادم وقاعدة البيانات، بالإضافة إلى تحديد الوظائف الأساسية التي يقدمها النظام وآلية تنفيذها. ويساعد هذا التحليل في بناء نظام منظم وقابل للتطوير، ويقلل من احتمالية حدوث الأخطاء أثناء مرحلة البرمجة.

اعتمد المشروع على بيئة تطوير تعتمد على **Node.js** و**Express.js** في الجانب الخلفي (Backend)، بينما تم استخدام **HTML** و**CSS** و**JavaScript** لبناء واجهة المستخدم (Frontend). وقد تم تصميم النظام وفق مبدأ فصل المسؤوليات (Separation of Concerns)، بحيث يؤدي كل جزء من أجزاء النظام وظيفة محددة ومستقلة.

أولاً: فكرة عمل النظام (System Overview)

يعتمد النظام على إدارة بيانات الطلاب بطريقة إلكترونية، حيث يستطيع المستخدم تنفيذ العمليات الأساسية من خلال واجهة استخدام بسيطة وسهلة، بينما يتولى الخادم معالجة الطلبات وتنفيذ العمليات المطلوبة.

يتكون النظام من عدة صفحات مترابطة، لكل صفحة وظيفة محددة، ويتم التنقل بينها من خلال الصفحة الرئيسية.

وتتمثل الوظائف الأساسية للنظام في:

- إضافة طالب جديد.
- البحث عن طالب.
- تعديل بيانات الطالب.
- عرض جميع الطلاب.
- عرض التقارير والإحصائيات الأساسية.

وتعمل هذه الوظائف بصورة متكاملة لتوفير نظام بسيط وفعال لإدارة درجات الطلاب.

ثانياً: البيئة التقنية للنظام (Technical Environment)

تم تطوير النظام باستخدام مجموعة من التقنيات الحديثة التي تتكامل مع بعضها البعض لتوفير بيئة تطوير مستقرة وسهلة الصيانة.

1. Node.js

يُستخدم Node.js كبيئة تشغيل لتطبيق JavaScript على الخادم، حيث يقوم باستقبال طلبات المستخدم ومعالجتها وإرسال النتائج المناسبة.

ويتميز Node.js بالأداء العالي واعتماده على نموذج البرمجة غير المتزامنة (Asynchronous Programming)، مما يجعله مناسبًا لتطوير تطبيقات الويب.

---

2. Express.js

يُعد Express.js إطار عمل مبسطًا يعمل فوق Node.js، ويستخدم لتنظيم مسارات التطبيق (Routes)، ومعالجة الطلبات (Requests)، وإرسال الاستجابات (Responses).

كما يساعد على تنظيم المشروع وتقليل حجم الكود البرمجي.

---

3. HTML

تم استخدام HTML لبناء الصفحات المختلفة للنظام مثل:

- الصفحة الرئيسية.
- صفحة إضافة الطالب.
- صفحة البحث.
- صفحة تعديل البيانات.
- صفحة عرض الطلاب.
- صفحة التقارير.

---

4. CSS

استخدمت CSS لتنسيق صفحات النظام وتحسين تجربة المستخدم، حيث تم تصميم واجهات بسيطة وسهلة الاستخدام مع مراعاة ترتيب العناصر وتناسق الألوان.

---

5. JavaScript

تستخدم JavaScript في الجانب الأمامي للتعامل مع أحداث المستخدم مثل الضغط على الأزرار، والتحقق من صحة البيانات، وإرسال الطلبات إلى الخادم.

ثالثاً: البنية العامة للنظام (System Architecture)

تم تصميم المشروع باستخدام هيكلية متعددة الطبقات (Multi-Tier Architecture)، حيث تم فصل كل جزء من أجزاء النظام عن الآخر.

ويتكون النظام من أربع طبقات رئيسية:

1. طبقة العرض (Presentation Layer)

وهي الطبقة التي يتعامل معها المستخدم مباشرة.

وتشمل جميع صفحات HTML بالإضافة إلى ملفات CSS وJavaScript.

تقوم هذه الطبقة بما يلي:

- عرض البيانات.

- استقبال المدخلات.
- إرسال الطلبات للخادم.
- عرض نتائج العمليات.

2Business Logic Layer). طبقة منطق الأعمال (

تمثل هذه الطبقة قلب النظام.

وتوجد داخل ملفات JavaScript الموجودة في الخادم.

وهي المسؤولة عن:

- التحقق من البيانات.
- تنفيذ عمليات الإضافة.
- تنفيذ عمليات البحث.
- تعديل بيانات الطلاب.
- معالجة الأخطاء.

3Routing Layer). طبقة التوجيه (

تمثلها ملفات Express Router.

وتقوم باستقبال الطلبات القادمة من المستخدم ثم توجيهها إلى الوظيفة المناسبة داخل النظام.

ومن أهم المسارات الموجودة في المشروع:

- إضافة طالب.
- تعديل طالب.
- البحث عن طالب.
- عرض جميع الطلاب.

وقد تم فصل هذه المسارات داخل ملف مستقل مثل

/router

studentRoute.js

4Data Layer). طبقة البيانات (

تمثل الجزء المسؤول عن حفظ واسترجاع البيانات.

وتحتوي على بيانات الطلاب مثل:

- الاسم.
- رقم القيد.
- الدرجة.
- الرقم التعريفي.

وتتعامل هذه الطبقة مع عمليات القراءة والإضافة والتعديل بطريقة منظمة.

System Workflowرابعاً: دورة عمل النظام (

تمر البيانات داخل النظام بعدة مراحل حتى يتم تنفيذ العملية المطلوبة.

الخطوة الأولى

يقوم المستخدم بفتح الصفحة الرئيسية.

الخطوة الثانية

يختار العملية المطلوبة مثل:

- إضافة طالب.
- البحث.
- تعديل البيانات.

الخطوة الثالثة

يقوم المستخدم بإدخال البيانات داخل النموذج.

الخطوة الرابعة

تقوم JavaScript بالتحقق من صحة المدخلات.

مثلاً:

- هل الاسم فارغ؟
- هل رقم القيد صحيح؟
- هل الدرجة رقمية؟

الخطوة الخامسة

إذا كانت البيانات صحيحة، يتم إرسالها إلى خادم Express باستخدام طلب HTTP مناسب.

مثل:

- POST. للإضافة
- GET. للبحث أو العرض
- PUT. للتعديل
- DELETE. للحذف (إذا تمت إضافته مستقبلًا)

الخطوة السادسة

يقوم Express باستقبال الطلب ثم تحويله إلى ملف المسارات (Routes).

الخطوة السابعة

تنفذ الدالة المناسبة العملية المطلوبة على بيانات الطلاب.

الخطوة الثامنة

يرسل الخادم نتيجة العملية إلى واجهة المستخدم.

الخطوة التاسعة

تقوم الواجهة بعرض رسالة نجاح أو خطأ مع تحديث البيانات المعروضة للمستخدم.

خامساً: تحليل تدفق البيانات (Data Flow Analysis)

يتدفق انتقال البيانات داخل النظام وفق التسلسل التالي:

المستخدم ⟵ واجهة المستخدم (HTML/CSS/JavaScript) ⟵ Express Routes ⟵ منطق التطبيق (Business Logic) ⟵ طبقة البيانات ⟵ النتيجة ⟵ واجهة المستخدم.

ويضمن هذا التسلسل فصل كل مرحلة عن الأخرى، مما يسهل اختبار النظام وصيانته وإضافة خصائص جديدة دون التأثير على بقية الأجزاء.

سادساً: تحليل وظائف النظام (Functional Analysis)

يعتمد النظام على مجموعة من الوظائف الأساسية، ولكل وظيفة مدخلات ومعالجة ومخرجات.

1. إضافة طالب

المدخلات:

- اسم الطالب.
- رقم القيد.
- الدرجة.

المعالجة:

- التحقق من صحة البيانات.
- إنشاء سجل جديد.
- حفظ البيانات.

المخرجات:

- رسالة نجاح.
- تحديث قائمة الطلاب.

---

2. البحث عن طالب

المدخلات:

- الاسم أو رقم القيد.

المعالجة:

- البحث داخل بيانات الطلاب.
- مقارنة المدخلات بالسجلات الموجودة.

المخرجات:

- عرض بيانات الطالب.
- أو رسالة بعدم العثور على نتائج.

---

3. تعديل البيانات

المدخلات:

- بيانات الطالب الجديدة.

المعالجة:

- تحديد الطالب.
- تحديث السجل.
- حفظ التعديلات.

المخرجات:

- رسالة تؤكد نجاح عملية التعديل.

---

4. عرض جميع الطلاب

المدخلات:

- طلب عرض البيانات.

المعالجة:

- جلب جميع السجلات.
- ترتيبها.

المخرجات:

- جدول يحتوي على جميع بيانات الطلاب.

الفصل الثامن: تصميم النظام (System Design)

مقدمة

بعد الانتهاء من مرحلة تحليل النظام وتحديد احتياجات المستخدم والمتطلبات الوظيفية وغير الوظيفية، تأتي مرحلة **تصميم النظام (System Design)**، والتي تمثل الجسر الذي يربط بين التحليل والتنفيذ البرمجي.

في هذه المرحلة يتم تحديد الهيكل العام للنظام، وطريقة تفاعل المستخدم مع النظام، والعلاقات بين مكوناته المختلفة، بالإضافة إلى تصميم قاعدة البيانات وهيكل الكائنات البرمجية.

وقد تم تصميم مشروع **نظام إدارة درجات الطلاب** وفق مبادئ هندسة البرمجيات الحديثة، بحيث يكون النظام واضح البنية، سهل التطوير، وقابلًا للصيانة والتوسع مستقبلاً.

يعتمد تصميم المشروع على عدة مخططات أساسية، وهي:

- Use Case Diagram
- Class Diagram
- ER Diagram
- System Architecture Diagram
- Sequence Diagram (اختياري)
- Activity Diagram (اختياري)

ويساعد كل مخطط في توضيح جانب معين من النظام.

أولاً: معمارية النظام (System Architecture)

يعتمد المشروع على **Three-Tier Architecture**، وهي إحدى أكثر البنى استخدامًا في تطبيقات الويب الحديثة.

تنقسم المعمارية إلى ثلاث طبقات رئيسية

المستخدم

|

▼

واجهة المستخدم (Frontend)

HTML + CSS + JavaScript

|

▼

Express.js Application Server

Routes + Business Logic

|

▼

Data Storage Layer

Student Records / JSON

ة

الطبقة الأولى: واجهة المستخدم (Presentation Layer)

تمثل الجزء الذي يتفاعل معه المستخدم مباشرة.

تتكون من الصفحات التالية:

- index.html
- add.html
- edit.html
- students.html
- reports.html
- insertDataPage.html

وظيفتها:

- عرض النماذج.
- استقبال البيانات.
- إرسال الطلبات.
- عرض النتائج.

الطبقة الثانية: منطق التطبيق (Application Layer)

تمثل الجزء المسؤول عن تنفيذ جميع العمليات.

وتشمل:

- Express Server
- studentRoute.js
- Validation
- Business Logic

ومن أهم مسؤولياتها:

- استقبال الطلبات.
- التحقق من صحة البيانات.
- معالجة العمليات.
- إرسال النتائج.

الطبقة الثالثة: طبقة البيانات (Data Layer)

تمثل مكان تخزين بيانات الطلاب.

وتشمل:

- Student Object
- Student Array
- JSON Data

وتقوم بـ:

- إضافة السجلات.
- تحديث البيانات.

- حذف البيانات (مستقبلاً).
- قراءة البيانات.

لماذا اخترنا هذه المعمارية؟

لأنها تحقق:

- فصل المسؤوليات.
- سهولة الصيانة.
- سهولة الاختبار.
- إمكانية تطوير كل طبقة بشكل مستقل.
- تقليل الترابط بين أجزاء المشروع.

ثانياً: مخطط حالات الاستخدام (Use Case Diagram)

ما هو Use Case Diagram؟

هو مخطط يوضح جميع العمليات التي يستطيع المستخدم تنفيذها داخل النظام، والعلاقة بين المستخدم وهذه العمليات.

يعتبر من أهم مخططات UML لأنه يوضح وظائف النظام من وجهة نظر المستخدم.

الممثل الرئيسي (Actor)

في هذا المشروع يوجد ممثل واحد فقط وهو:

المستخدم (User)

وهو الشخص الذي يتعامل مع النظام لإدارة بيانات الطلاب.

حالات الاستخدام

يتفاعل المستخدم مع النظام من خلال الحالات التالية:

1. Add Student

يقوم المستخدم بإضافة طالب جديد.

المدخلات:

- الاسم
- رقم القيد
- الدرجة

النظام يقوم بـ:

- التحقق من صحة البيانات.
- حفظ الطالب.
- عرض رسالة نجاح.

---

2. Search Student

يقوم المستخدم بالبحث عن طالب.

يمكن البحث باستخدام:

- الاسم.
- رقم القيد.

النظام يقوم بمقارنة البيانات ثم يعرض النتائج.

---

3. Edit Student

يقوم المستخدم بتعديل بيانات الطالب.

النظام يقوم:

- بجلب بيانات الطالب.
- السماح بالتعديل.
- حفظ التعديلات.

---

4. View Students

يقوم المستخدم بعرض جميع الطلاب داخل جدول.

يعرض النظام:

- الاسم.
- رقم القيد.
- الدرجة.

---

5. View Reports

يعرض النظام تقارير وإحصائيات عامة عن الطلاب.

مثل:

- عدد الطلاب.
- متوسط الدرجات.
- أعلى درجة.
- أقل درجة.

وصف المخطط

يرتبط المستخدم بجميع حالات الاستخدام السابقة من خلال خطوط اتصال مباشرة، مما يدل على أن المستخدم هو المسؤول عن بدء جميع العمليات داخل النظام، بينما يقوم النظام بتنفيذ المعالجة وإرجاع النتائج.

ويعكس هذا التصميم بساطة المشروع وتركيزه على إدارة بيانات الطلاب دون الحاجة إلى وجود أكثر من نوع من المستخدمين في الإصدار الحالي.

ثالثاً: مخطط الفئات (Class Diagram)

مقدمة

يستخدم مخطط الفئات لتوضيح الكائنات البرمجية الموجودة داخل النظام والعلاقات بينها.

يساعد هذا المخطط في تحويل التصميم إلى كود برمجي.

الفئة الرئيسية

Student

تمثل الطالب داخل النظام.

وتحتوي على الخصائص التالية

الوصف	النوع	الخاصية
معرف الطالب	Number	id
اسم الطالب	String	name
رقم القيد	String	reg_number
الدرجة	Number	degree

الدوال الخاصة بالفئة

addStudent()

وظيفتها:

إضافة طالب جديد.

---

updateStudent()

وظيفتها:

تعديل بيانات الطالب.

---

searchStudent()

وظيفتها:

البحث عن طالب.

---

loadStudents()

وظيفتها:

تحميل جميع الطلاب.

---

deleteStudent()

(يمكن إضافتها مستقبلاً)

وظيفتها:

حذف طالب.

العلاقات

لا يحتوي المشروع الحالي على أكثر من Class رئيسية.

لكن يمكن مستقبلاً إضافة:

Student

↓

Course

↓

Department

↓

Instructor

↓

Report

وبذلك يصبح النظام أكثر احترافية.

رابعاً: مخطط قاعدة البيانات (ER Diagram)

يعتمد المشروع على كيان رئيسي واحد في الإصدار الحالي.

الكيان

Student

الخصائص

StudentID

المفتاح الأساسي (Primary Key).

StudentName

اسم الطالب.

RegistrationNumber

رقم القيد.

Degree

درجة الطالب.

الشكل المنطقي:

```
+----------------------+
|        Student       |
+----------------------+
|        StudentID (PK) |
|                 Name |
|    RegistrationNumber |
|                Degree |
+----------------------+
```

ن

يمثل هذا الجدول جميع الطلاب الموجودين داخل النظام.

لكل طالب معرف فريد يمنع تكرار البيانات.

ويتم استخدام هذا المعرف عند تنفيذ عمليات:

- البحث.
- التعديل.
- الحذف.

خامساً: تدفق العمليات داخل النظام (Workflow)

تمر البيانات داخل المشروع وفق الخطوات التالية:

1. يفتح المستخدم الصفحة الرئيسية.

↓

2. يختار العملية المطلوبة.

↓

3. ينتقل إلى الصفحة المناسبة.

↓

4. يدخل البيانات.

↓

5. تتحقق JavaScript من صحة المدخلات.

↓

6. يتم إرسال الطلب إلى Express Server.

↓

7. يقوم Router بتحديد الدالة المناسبة.

↓

8. تنفذ العملية.

↓

9. يتم تحديث البيانات.

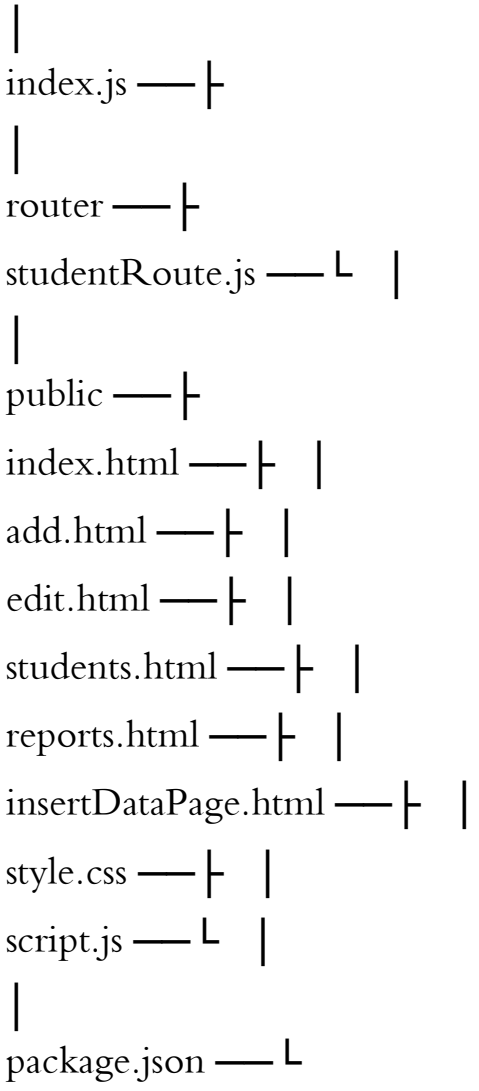
↓

10. إرسال النتيجة إلى المستخدم.

سادساً: هيكل ملفات المشروع (Project Structure)

تم تنظيم المشروع بطريقة تحقق مبدأ **Modularity** وفصل المسؤوليات، بحيث يكون لكل ملف وظيفة محددة داخل النظام

Student Management System



شرح الهيكل

- **index.js**: نقطة تشغيل التطبيق وتهيئة خادم Express.
- **studentRoute.js**: يحتوي على جميع مسارات (Routes) الخاصة بعمليات الطلاب.
- **public**: يضم صفحات الواجهة الأمامية وملفات التنسيق والبرمجة الخاصة بالعميل.
- **package.json**: يدير مكتبات المشروع وإعداداته.

الفصل التاسع: المحاضرة الأولى – مقدمة في بناء البرمجيات (Introduction to Software Construction)

مقدمة

تُعد مرحلة بناء البرمجيات (Software Construction) المرحلة التي يتم فيها تحويل التصميم النظري إلى تطبيق عملي قابل للتنفيذ. ففي هذه المرحلة يبدأ المطور بكتابة الكود البرمجي وفق تصميم محدد، مع مراعاة جودة الكود وتنظيمه وقابليته للصيانة والتطوير.

في مشروع **نظام إدارة درجات الطلاب** لم يكن الهدف مجرد إنشاء صفحات ويب تعمل بشكل صحيح، وإنما كان التركيز على بناء نظام منظم يعتمد على مبادئ هندسة البرمجيات الحديثة. لذلك تم تحسين المشروع تدريجيًا بدءًا من نسخة أولية بسيطة وحتى أصبح مشروعًا منظمًا يعتمد على فصل المسؤوليات، وإعادة استخدام الكود، ووضوح التسمية، وتقليل الترابط بين المكونات.

سيتم في هذا الفصل توضيح كيفية تطبيق أهم مفاهيم المحاضرة الأولى على المشروع.

أولاً: تحويل التصميم إلى بناء (Transition from Design to Construction)

المفهوم

يقصد بتحويل التصميم إلى بناء عملية تحويل المخططات التي تم إعدادها أثناء مرحلة التصميم (مثل Use Case وClass Diagram وER Diagram) إلى كود برمجي فعلي يعمل داخل النظام.

ففي مرحلة التصميم تم تحديد أن النظام يحتاج إلى عمليات إضافة طالب، والبحث عنه، وتعديل بياناته، وعرض جميع الطلاب. أما في مرحلة البناء فقد تم تنفيذ هذه العمليات باستخدام JavaScript وNode.js وExpress.js.

قبل التحسين

في النسخة الأولى من المشروع، كانت جميع العمليات موجودة داخل ملف واحد، ولم يكن هناك تنظيم واضح بين وظائف النظام

}app.post("/add", function(req,res)

```
// حفظ البيانات

});

}app.get("/students", function(req,res)

// عرض الطلاب

});

}app.put("/edit", function(req,res)

// تعديل الطالب

});
```

كان هذا الأسلوب مناسبًا للمشاريع الصغيرة جدًا، لكنه يؤدي إلى تضخم الملف الرئيسي وصعوبة فهم الكود مع زيادة حجم المشروع.

بعد التحسين

تم فصل العمليات داخل Router مستقل

```
;router.post("/addStudent")

;router.get("/students")

;router.put("/updateStudent")

;router.get("/searchStudent")
```

وأصبحت جميع العمليات الخاصة بالطلاب داخل ملف:

studentRoute.js

بينما بقي الملف الرئيسي **index.js** مسؤولًا فقط عن تشغيل الخادم وربط المسارات.

الفوائد

حقق هذا التغيير عددًا من المزايا المهمة، منها:

- تقليل حجم الملف الرئيسي.
- سهولة الوصول إلى الوظائف الخاصة بالطلاب.
- إمكانية إضافة وظائف جديدة دون التأثير على باقي النظام.
- تحسين تنظيم المشروع.

ثانياً: فصل المسؤوليات (Separation of Concerns)

المفهوم

يقصد بفصل المسؤوليات أن يكون لكل جزء من النظام وظيفة محددة لا يتجاوزها، بحيث لا يتم خلط واجهة المستخدم مع منطق التطبيق أو مع إدارة البيانات.

التطبيق في المشروع

تم تقسيم المشروع إلى عدة أجزاء مستقلة:

index.js

مسؤول عن:

- تشغيل الخادم.
- إعداد Express.
- ربط ملفات Routes.

ولا يحتوي على منطق الأعمال الخاص بالطلاب.

---

studentRoute.js

مسؤول عن:

- إضافة طالب.
- البحث.
- التعديل.
- عرض الطلاب.

HTML

مسؤولة فقط عن:

- عرض الصفحات.
- إدخال البيانات.

ولا تحتوي على معالجة البيانات.

CSS

مسؤولة عن:

- تصميم الصفحات.
- تنسيق الأزرار.
- تحسين واجهة المستخدم.

JavaScript

مسؤولة عن:

- استقبال المدخلات.
- التحقق من البيانات.
- إرسال الطلبات للخادم.
- عرض النتائج.

الفائدة

أصبح كل ملف يؤدي مهمة واحدة فقط، مما يجعل المشروع أكثر وضوحًا وأسهل في الصيانة والتطوير.

ثالثاً: مبدأ عدم التكرار (DRY – Don’t Repeat Yourself)

المفهوم

ينص مبدأ DRY على أنه يجب عدم تكرار نفس الكود في أكثر من مكان داخل المشروع، بل يجب كتابة الكود مرة واحدة وإعادة استخدامه عند الحاجة.

قبل التحسين

كان التحقق من بيانات الطالب يُكتب في أكثر من صفحة

```
if(name==""){

alert("Name Required");

}

if(reg==""){

alert("Registration Required");
```

}وكان نفس الكود يتكرر أيضًا في صفحة التعديل.

بعد التحسين

تم إنشاء دالة مشتركة

```
function validateStudent(){

...

}
```

ثم يتم استدعاؤها عند الحاجة

```
validateStudent();
```

الفائدة

- تقليل حجم الكود.
- سهولة تعديل التحقق من البيانات.
- تقليل الأخطاء.
- تحسين قابلية الصيانة.

رابعاً: الكود النظيف (Clean Code)

المفهوم

الكود النظيف هو الكود الذي يستطيع أي مطور فهمه بسهولة دون الحاجة إلى شرح إضافي.

ويتميز بأنه:

- واضح.
- بسيط.
- منظم.
- سهل القراءة.

قبل التحسين:

```
function a(){

}

let x;

let y;
```

هذه الأسماء لا توضح وظيفة الدالة أو المتغير.

بعد التحسين:

```
function addStudent(){

}

function updateStudent(){

}

function searchStudent(){
```

}

let studentName;

let registrationNumber;

let studentDegree;

الفائدة

- فهم أسرع للكود.
- تقليل الأخطاء.
- سهولة التعاون بين أعضاء الفريق.
- سهولة الصيانة.

خامساً: التغليف (Encapsulation)

المفهوم

التغليف هو تجميع البيانات والعمليات الخاصة بها داخل وحدة واحدة، مع منع الوصول المباشر إلى التفاصيل الداخلية إلا من خلال واجهات محددة.

التطبيق في المشروع

بدلاً من السماح لأي ملف بالتعامل مباشرة مع بيانات الطلاب، تم وضع جميع العمليات داخل ملف

studentRoute.js

ولا يستطيع أي جزء آخر من النظام تعديل بيانات الطلاب إلا عن طريق الدوال الموجودة داخل هذا الملف

على سبيل المثال:

addStudent()

updateStudent()

()searchStudent

تتولى هذه الدوال جميع عمليات معالجة البيانات، بينما لا تتعامل صفحات الواجهة مباشرة مع تفاصيل التخزين أو التنفيذ.

الفوائد

- حماية البيانات.
- تقليل الأخطاء الناتجة عن التعديل المباشر.
- تنظيم الوصول إلى البيانات.
- تسهيل اختبار النظام وصيانته.

سادساً: إعادة استخدام الكود (Code Reusability)

المفهوم

تعني إعادة استخدام الكود كتابة الدوال أو الوحدات البرمجية بطريقة تسمح باستخدامها في أكثر من جزء داخل المشروع دون الحاجة إلى إعادة كتابتها.

التطبيق في المشروع

تم إنشاء دوال عامة مثل:

;()loadStudents

;()searchStudent

;()validateStudent

وأصبحت هذه الدوال تُستخدم في أكثر من صفحة، مثل صفحة عرض الطلاب وصفحة التعديل، بدلاً من كتابة منطق متكرر في كل صفحة.

الفوائد

- تقليل حجم المشروع.
- سهولة الصيانة.
- تقليل نسبة الأخطاء.
- تسريع عملية التطوير.

سابعاً: نتائج تطبيق مفاهيم المحاضرة الأولى

بعد تطبيق مفاهيم **مقدمة في بناء البرمجيات** على مشروع **نظام إدارة درجات الطلاب**، ظهرت مجموعة من التحسينات الواضحة، أهمها:

بعد التحسين	قبل التحسين
فصل العمليات إلى ملفات مستقلة	جميع العمليات داخل ملف واحد
إنشاء دوال مشتركة وإعادة استخدامها	تكرار الكود في أكثر من مكان
استخدام أسماء معبرة وواضحة	أسماء متغيرات ودوال غير واضحة
فصل الواجهة عن منطق الأعمال	خلط بين واجهة المستخدم ومنطق التطبيق
سهولة التوسع وإضافة وظائف جديدة	صعوبة إضافة ميزات جديدة

الفصل العاشر: المحاضرة الثانية – المبادئ الأساسية في بناء البرمجيات (Software Construction Principles)

مقدمة

تعتمد البرمجيات الحديثة على مجموعة من المبادئ الأساسية التي تهدف إلى إنتاج أنظمة ذات جودة عالية، يسهل فهمها وصيانتها وتطويرها مستقبلاً. فنجاح أي مشروع برمجي لا يقاس فقط بقدرته على تنفيذ الوظائف المطلوبة، بل يقاس أيضًا بمدى تنظيم الكود، وسهولة إضافة ميزات جديدة، وتقليل الترابط بين أجزاء النظام.

في مشروع **نظام إدارة درجات الطلاب** تم تطبيق أربعة مبادئ رئيسية من مبادئ بناء البرمجيات، وهي:

- التجريد (Abstraction).
- الوحدات النمطية (Modularity).
- التغليف (Encapsulation).
- إخفاء المعلومات (Information Hiding).

وقد ساهم تطبيق هذه المبادئ في جعل المشروع أكثر تنظيمًا ومرونة، كما وفر بيئة مناسبة للتوسع المستقبلي دون الحاجة إلى إعادة بناء النظام.

أولاً: التجريد (Abstraction)

مفهوم التجريد

التجريد هو إظهار الوظائف الأساسية التي يحتاجها المستخدم، مع إخفاء التفاصيل الداخلية الخاصة بطريقة التنفيذ. وبذلك يتعامل المستخدم مع النظام من خلال واجهات واضحة وبسيطة، دون الحاجة إلى معرفة كيفية تنفيذ العمليات داخل الخادم.

تطبيق التجريد في المشروع

في نظام إدارة درجات الطلاب، لا يحتاج المستخدم إلى معرفة كيفية حفظ البيانات أو طريقة تنفيذ عمليات البحث والتعديل، وإنما يتعامل مع أزرار ونماذج واضحة مثل:

- إضافة طالب.
- تعديل طالب.
- البحث عن طالب.
- عرض الطلاب.

أما العمليات البرمجية التي تنفذ هذه الوظائف فهي مخفية داخل الخادم.

قبل التحسين

كانت بعض العمليات البرمجية مكتوبة مباشرة داخل الصفحة، مما يجعل الواجهة تحتوي على تفاصيل تنفيذية لا تهم المستخدم

function save(){

// جميع خطوات التنفيذ هنا

}

بعد التحسين

تم إنشاء دوال واضحة تمثل الوظائف فقط

addStudent()

searchStudent()

updateStudent()

()loadStudents

كل دالة تستدعي المنطق الداخلي المناسب دون كشف تفاصيل التنفيذ.

فوائد تطبيق التجريد

حقق تطبيق التجريد عدة فوائد، أهمها:

- تبسيط استخدام النظام.
- فصل واجهة المستخدم عن التنفيذ الداخلي.
- تسهيل تعديل الكود دون التأثير على المستخدم.
- تحسين قابلية الصيانة.

ثانياً: الوحدات النمطية (Modularity)

مفهوم الوحدات النمطية

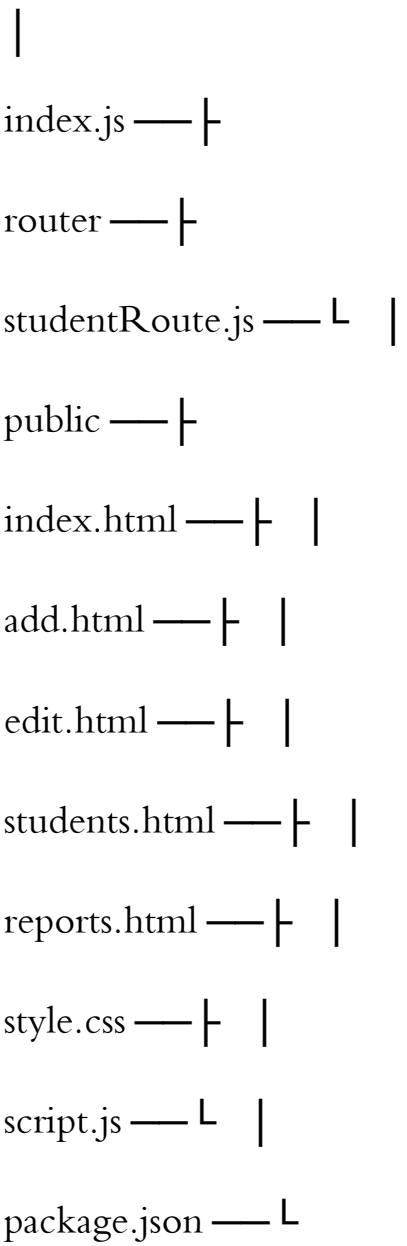
يقصد بالوحدات النمطية تقسيم المشروع إلى أجزاء مستقلة، بحيث تكون لكل وحدة مسؤولية محددة، ويمكن تطويرها أو تعديلها بصورة منفصلة.

ويعد هذا المبدأ من أهم مبادئ بناء البرمجيات لأنه يمنع تضخم الملفات ويسهل التعاون بين المطورين.

تطبيق الوحدات النمطية في المشروع

تم تقسيم المشروع إلى عدة وحدات مستقلة كما يلي:

Student Management System



index.js

مسؤول عن تشغيل التطبيق وربط الوحدات معًا.

studentRoute.js

يحتوي على جميع العمليات الخاصة بالطلاب.

HTML

تعرض الصفحات للمستخدم.

CSS

تنسق شكل الواجهة.

JavaScript

تتعامل مع المدخلات والتفاعل مع المستخدم.

Modularityقبل تطبيق

كانت جميع العمليات داخل ملف واحد.

نتج عن ذلك:

- زيادة حجم الملف.
- صعوبة قراءة الكود.
- صعوبة إضافة ميزات جديدة.
- زيادة احتمال حدوث الأخطاء.

بعد التطبيق

أصبح لكل ملف مسؤولية واحدة فقط، وأصبحت عملية الصيانة أسهل بكثير.

فوائد الوحدات النمطية

- سهولة التطوير.
- تقليل الأخطاء.
- سهولة اختبار النظام.
- تحسين تنظيم المشروع.
- إمكانية توزيع العمل بين أعضاء الفريق.

ثالثاً: التغليف (Encapsulation)

مفهوم التغليف

التغليف يعني جمع البيانات والعمليات المتعلقة بها داخل وحدة واحدة، ومنع الوصول المباشر إليها إلا من خلال واجهات محددة.

ويهدف هذا المبدأ إلى حماية البيانات ومنع التعديل غير الصحيح عليها.

تطبيق التغليف في المشروع

تم تجميع جميع العمليات الخاصة بالطلاب داخل ملف:

studentRoute.js

ويحتوي على دوال مثل:

addStudent()

updateStudent()

searchStudent()

loadStudents()

ولا يسمح لبقية أجزاء النظام بالتعامل مع بيانات الطلاب بصورة مباشرة.

مثال

بدلاً من أن تقوم صفحة HTML بحفظ الطالب بنفسها، فإنها ترسل البيانات إلى:

router.post("/addStudent")

فوائد التغليف

- حماية البيانات.
- تقليل الأخطاء.
- تنظيم الكود.
- سهولة اختبار النظام.
- سهولة إضافة قواعد تحقق جديدة.

رابعاً: إخفاء المعلومات (Information Hiding)

مفهوم إخفاء المعلومات

يعني إخفاء التفاصيل الداخلية للنظام عن بقية الأجزاء، بحيث يتم التعامل مع الوظائف من خلال واجهات واضحة دون معرفة كيفية تنفيذها.

تطبيق المبدأ في المشروع

المستخدم يرى فقط:

- نموذج إضافة طالب.
- زر الحفظ.
- جدول الطلاب.

ولا يعلم شيئًا عن:

- طريقة تخزين البيانات.
- طريقة تنفيذ Routes.
- طريقة معالجة الطلبات.
- التحقق من صحة البيانات.

كما أن ملفات HTML لا تتعامل مباشرة مع البيانات، وإنما ترسل الطلبات إلى Express.

مثال

بدلاً من كتابة جميع العمليات داخل الصفحة:

saveStudent();

تقوم الصفحة بإرسال الطلب فقط.

أما التنفيذ الحقيقي فيوجد داخل:

studentRoute.js

Information Hiding

- زيادة الأمان.
- تقليل الترابط بين الملفات.
- سهولة تغيير طريقة التنفيذ.
- سهولة استبدال قاعدة البيانات مستقبلًا.

خامساً: العلاقة بين المبادئ الأربعة

عملت المبادئ السابقة معًا لتكوين مشروع أكثر تنظيمًا:

- أخفى تفاصيل التنفيذ عن المستخدم. **Abstraction**
- قسم المشروع إلى وحدات مستقلة. **Modularity**
- جمع العمليات المتعلقة بالطلاب داخل وحدة واحدة. **Encapsulation**
- أخفى تفاصيل التنفيذ الداخلية عن بقية النظام. **Information Hiding**

والنتيجة كانت مشروعًا أكثر مرونة وقابلية للتطوير.

سادساً: مقارنة قبل وبعد تطبيق المبادئ:

بعد التطبيق	قبل التطبيق
تقسيم المشروع إلى وحدات مستقلة	ملف واحد يحتوي جميع العمليات
فصل واضح بين الطبقات	تداخل بين الواجهة ومنطق التطبيق
إمكانية تعديل كل جزء بشكل مستقل	سهولة كسر النظام عند التعديل
سهولة التوسع وإضافة وظائف مستقبلية	صعوبة إضافة ميزات جديدة
ترابط منخفض وتنظيم أفضل	ترابط مرتفع بين الملفات

الفصل الحادي عشر: المحاضرة الثالثة – كتابة الكود النظيف (Clean Code)

مقدمة

تُعد كتابة الكود النظيف (Clean Code) من أهم المهارات التي يجب أن يمتلكها مطور البرمجيات، حيث لا يكفي أن يكون البرنامج قادرًا على تنفيذ الوظائف المطلوبة، بل يجب أن يكون الكود نفسه واضحًا ومنظمًا وسهل الفهم والصيانة.

في المشاريع البرمجية الواقعية، لا يعمل المطور بمفرده، وإنما يشارك مع فريق من المطورين، لذلك يجب أن يكون الكود مكتوبًا بطريقة يستطيع أي مطور آخر فهمها بسهولة دون الحاجة إلى شرح مفصل. ولهذا السبب تركز مادة بناء البرمجيات على مبادئ الكود النظيف، لأنها تؤثر مباشرة في جودة المشروع وقابليته للتطوير.

في مشروع **نظام إدارة درجات الطلاب** تم تطبيق عدد من مبادئ الكود النظيف أثناء إعادة هيكلة المشروع، مما ساهم في تحسين جودة الكود وتقليل الأخطاء وزيادة قابلية الصيانة.

سيتم في هذا الفصل توضيح أهم هذه المبادئ مع أمثلة عملية من المشروع.

أولاً: التسمية الجيدة (Good Naming)

مفهوم التسمية الجيدة

تُعد تسمية المتغيرات والدوال والملفات من أهم عناصر الكود النظيف، لأن الاسم هو أول ما يقرأه المطور لفهم وظيفة الكود. يجب أن يكون الاسم معبرًا عن الغرض الحقيقي من العنصر البرمجي، وأن يكون واضحًا دون الحاجة إلى تعليق يشرح معناه.

الاسم الجيد يجيب مباشرة عن سؤال: **ما وظيفة هذا المتغير أو هذه الدالة؟**

التطبيق في المشروع

في النسخة الأولى من المشروع كانت بعض أسماء المتغيرات والدوال قصيرة وغير واضحة.

قبل التحسين:

```
}()function a

{

;let x

;let y

; let z
```

هذه الأسماء لا توضح وظيفة الدالة أو المتغير، مما يجعل قراءة الكود صعبة، خاصة إذا كان المشروع كبيرًا أو يعمل عليه أكثر من مطور.

بعد التحسين

تم استبدال الأسماء العامة بأسماء تعبر عن الوظيفة الحقيقية

```
}()function addStudent

{

}()function updateStudent

{

}()function searchStudent

{

;let studentName

;let registrationNumber

;let studentDegree
```

أصبحت أسماء الدوال والمتغيرات تصف بوضوح الوظيفة التي تؤديها، مما يجعل الكود أكثر وضوحًا.

أمثلة أخرى من المشروع

بدلاً من:

;let data

تم استخدام

;let studentData

وبدلاً من

;let result

تم استخدام

;let searchResult

ة

ساهمت التسمية الواضحة في:

- تسهيل قراءة الكود.
- تقليل الحاجة إلى التعليقات.
- تسريع فهم المشروع.
- تسهيل مراجعة الكود (Code Review).
- تقليل احتمالية الأخطاء.

ثانياً: الوضوح (Clarity)

مفهوم الوضوح

الوضوح يعني أن يكون الكود مقروءًا وكأنه جملة مفهومة، بحيث يستطيع المطور فهم ما يحدث دون الحاجة إلى تحليل طويل.

ويتحقق ذلك من خلال:

- كتابة تعليمات بسيطة.
- تجنب الاختصارات غير المفهومة.
- تقسيم العمليات الكبيرة إلى خطوات صغيرة.
- استخدام أسماء واضحة.

التطبيق في المشروع

قبل التحسين

```
function p(){

console.log(n);

}
```

من الصعب معرفة وظيفة هذه الدالة أو معنى المتغير n.

بعد التحسين

```
function displayStudentDetails(student){

console.log(student.name);

console.log(student.reg_number);
```

```
console.log(student.degree);

}
```

أصبح الهدف من الدالة واضحًا بمجرد قراءة اسمها، كما أن أسماء المتغيرات تشرح نفسها.

أثر الوضوح على المشروع

ساعدت هذه التغييرات في:

- تسهيل صيانة المشروع.
- تقليل الوقت اللازم لفهم الكود.
- تسهيل اكتشاف الأخطاء.
- تحسين التعاون بين أعضاء الفريق.

ثالثاً: البساطة (Simplicity)

مفهوم البساطة

ينص هذا المبدأ على أن الحل الأبسط غالبًا يكون الأفضل، ويجب تجنب التعقيد غير الضروري في كتابة الكود.

الكود البسيط يكون:

- أقصر.
- أوضح.
- أسهل في الاختبار.
- أقل عرضة للأخطاء.

التطبيق في المشروع

قبل التحسين

```
if(student != null){

if(student.degree >= 50){

return true;

}

else{

return false;

}

}

else{

return false;

}
```

هذا الكود يحتوي على شروط متداخلة يصعب قراءتها.

بعد التحسين

```
return student != null && student.degree >= 50;
```

أصبحت نفس العملية تنفذ بسطر واحد واضح وسهل الفهم

ة

- تقليل التعقيد.
- تقليل الأخطاء المنطقية.
- سهولة الاختبار.
- سهولة الصيانة.

رابعاً: الالتزام بالمعايير (Consistency)

مفهوم الالتزام بالمعايير

الالتزام بالمعايير يعني استخدام أسلوب موحد في جميع أجزاء المشروع، سواء في تسمية الدوال أو تنسيق الكود أو تنظيم الملفات.

فعندما يستخدم جميع المطورين نفس القواعد، يصبح المشروع أكثر وضوحًا واحترافية.

التطبيق في المشروع

تم اعتماد أسلوب **camelCase** في تسمية المتغيرات والدوال

قبل التحسين

```
create_student();
```

search_student();

UpdateStudent();

توجد ثلاثة أساليب مختلفة للتسمية.

بعد التحسين

addStudent();

searchStudent();

updateStudent();

loadStudents();

أصبحت جميع الدوال تستخدم نفس النمط

تنظيم الملفات

تم أيضًا توحيد طريقة تنظيم المشرو

router/

public/

index.js

package.json

بدلاً من وضع جميع الملفات في المجلد الرئيسي

ر

- سهولة قراءة المشروع.
- توحيد أسلوب البرمجة.
- تقليل الأخطاء.
- تسهيل التعاون بين المطورين.

خامساً: تقليل التعليقات غير الضرورية

المفهوم

في الكود النظيف، يفضل أن يكون الكود واضحًا بذاته، بحيث لا يحتاج إلى عدد كبير من التعليقات لتفسيره.

قبل التحسين

// هذه الدالة تضيف طالباً

function addStudent(){

...

}

إذا كان اسم الدالة واضحًا، فإن التعليق لا يضيف قيمة.

بعد التحسين

function addStudent(){

…

}

متى نستخدم التعليقات؟

تستخدم التعليقات فقط عندما يكون هناك:

- خوارزمية معقدة.
- سبب تقني غير واضح.
- تحذير مهم للمطورين.

سادساً: كتابة دوال صغيرة (Small Functions)

المفهوم

يفضل أن تؤدي كل دالة مهمة واحدة فقط.

قبل التحسين

function processStudent(){

// التحقق

// الحفظ

// البحث

// التعديل

// الطباعة

}

الدالة تقوم بخمس مهام مختلفة.

بعد التحسين

validateStudent();

addStudent();

searchStudent();

updateStudent();

displayStudents();

أصبحت كل دالة مسؤولة عن عملية واحدة، مما يجعل المشروع أكثر تنظيمًا.

سابعاً: تجنب تكرار الكود

تم تطبيق مبدأ **DRY** من خلال إنشاء دوال عامة مثل

validateStudent();

loadStudents();

searchStudent();

واستخدامها في أكثر من صفحة، بدلاً من إعادة كتابة نفس المنطق.

ثامناً: أثر تطبيق الكود النظيف على المشروع

بعد تطبيق مبادئ الكود النظيف، تحسن المشروع بشكل ملحوظ، ويمكن تلخيص ذلك في الجدول التالي:

بعد التطبيق	قبل التطبيق
أسماء معبرة	أسماء غير واضحة
دوال قصيرة وواضحة	دوال طويلة ومعقدة
إعادة استخدام الدوال	تكرار في الكود
أسلوب موحد في التسمية	تنسيق غير موحد
سهولة القراءة والصيانة	صعوبة فهم المشروع

الفصل الثاني عشر: المحاضرة الرابعة – أنماط التصميم (Design Patterns)

مقدمة

مع ازدياد حجم المشاريع البرمجية، يصبح تنظيم الكود والمحافظة على قابليته للتطوير من أكبر التحديات التي تواجه المطورين. لذلك ظهرت **أنماط التصميم (Design Patterns)**، وهي حلول برمجية مجربة تُستخدم لمعالجة المشكلات المتكررة في تصميم البرمجيات.

لا تعتبر أنماط التصميم أكوادًا جاهزة، وإنما هي أساليب لتنظيم المشروع بطريقة تجعل الكود أكثر وضوحًا ومرونة وقابلية للصيانة. وقد تم تطبيق عدد من الأنماط المناسبة لطبيعة مشروع **نظام إدارة درجات الطلاب**، بينما لم يتم استخدام الأنماط التي لا تتناسب مع حجمه الحالي.

أولاً: Router Pattern

مفهوم النمط

يُستخدم **Router Pattern** لفصل مسارات التطبيق (Routes) عن الملف الرئيسي، بحيث لا يحتوي الملف الرئيسي على جميع العمليات البرمجية.

في تطبيقات Express.js يعد هذا النمط من أكثر الأنماط استخدامًا لأنه يساعد على تنظيم المشروع وتقليل حجم الملف الرئيسي.

قبل التطبيق

في النسخة الأولى كانت جميع المسارات موجودة داخل الملف الرئيسي

```
app.get("/students")
```

```
app.post("/add")
```

```
app.put("/edit")
```

```
app.get("/search")
```

ومع زيادة عدد الوظائف أصبح الملف الرئيسي كبيرًا ويصعب صيانته.

بعد التطبيق

تم إنشاء مجلد مستقل باسم

router/

وبداخله ملف

studentRoute.js

وأصبحت جميع العمليات الخاصة بالطلاب موجودة داخله.

أما الملف الرئيسي فأصبح مسؤولًا فقط عن:

- تشغيل الخادم.
- إعداد Express.
- استدعاء ملفات Router.

فوائد Router Pattern

- تقليل حجم الملف الرئيسي.
- سهولة إضافة Routes جديدة.
- تحسين تنظيم المشروع.
- تسهيل اختبار النظام.

ثانياً: Modular Design Pattern

مفهوم النمط

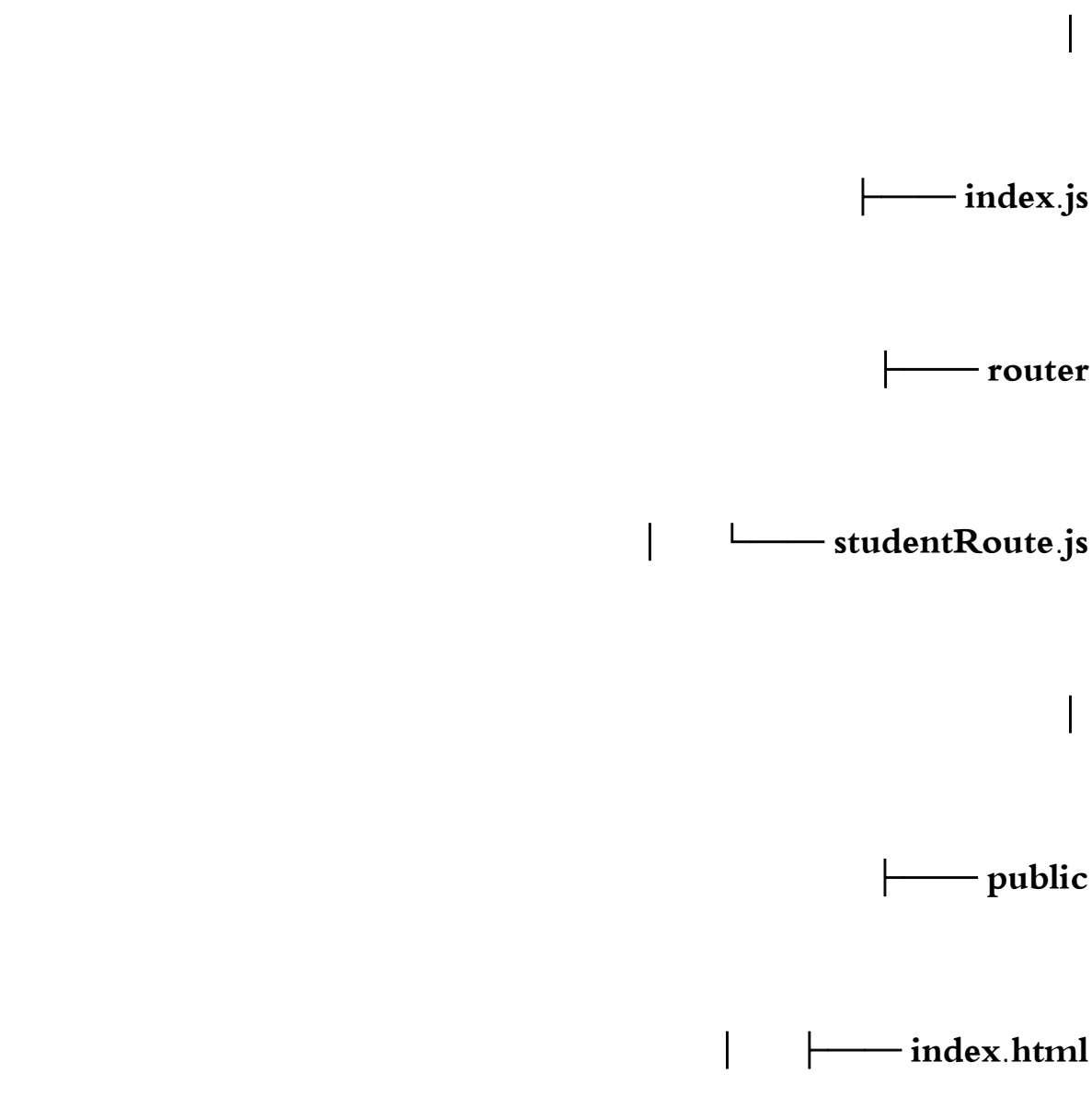
يقوم هذا النمط على تقسيم المشروع إلى وحدات مستقلة، بحيث تكون لكل وحدة وظيفة محددة.

بدلاً من وجود ملف ضخم يحتوي على جميع أجزاء المشروع، يتم توزيع المسؤوليات على عدة ملفات.

التطبيق في المشروع

تم تقسيم المشروع بالشكل التالي

Student Management System



add.html ├──── │

edit.html ├──── │

students.html ├──── │

reports.html ├──── │

style.css ├──── │

script.js └──── │

│

package.json └────

د

- سهولة التطوير.
- سهولة الصيانة.
- تقليل الترابط.
- إمكانية إعادة استخدام الملفات.

ثالثاً: Facade Pattern

مفهوم النمط

يوفر **Facade Pattern** واجهة بسيطة للتعامل مع مجموعة من العمليات المعقدة.

بدلاً من أن يستدعي المستخدم عدة عمليات مختلفة، يتم استدعاء دالة واحدة تتولى تنفيذ جميع الخطوات.

التطبيق في المشروع

عند إضافة طالب جديد تمر العملية بعدة خطوات:

1. التحقق من صحة البيانات.
2. إنشاء كائن الطالب.
3. حفظ البيانات.
4. تحديث قائمة الطلاب.
5. عرض رسالة النجاح.

بدلاً من استدعاء كل خطوة يدويًا، يتم استدعاء

addStudent();

الفوائد

- تبسيط الكود.
- تقليل التعقيد.
- سهولة الاستخدام.
- سهولة الصيانة.

Singleton Patternرابعاً:

مفهوم النمط

يضمن وجود نسخة واحدة فقط من كائن معين داخل النظام.

التطبيق في المشروع

في مشروع Node.js يوجد خادم Express واحد فقط.

بدلاً من إنشاء أكثر من Server داخل المشروع، يتم إنشاء نسخة واحدة فقط

```
const app = express();
```

.

الفوائد

- تقليل استهلاك الذاكرة.
- منع إنشاء نسخ متعددة.
- تحسين الأداء.
- سهولة إدارة التطبيق.

Factory Patternخامساً:

مفهوم النمط

يستخدم Factory لإنشاء الكائنات بطريقة منظمة دون أن يعتمد باقي المشروع على تفاصيل عملية الإنشاء.

التطبيق في المشروع

عند إنشاء طالب جديد لا يحتاج المستخدم إلى معرفة كيفية إنشاء كائن Student.

يكفي أن يرسل البيانات:

- الاسم.
- رقم القيد.
- الدرجة.

ويتولى النظام إنشاء الكائن المناسب.

قبل التطبيق

```
let student = {

id:id,

name:name,

degree:degree

}
```

بعد التطبيق

```
createStudent();
```

مقارنة قبل وبعد

بعد التطوير	قبل التطوير
studentRoute.js داخل Routes	جميع Routes داخل index.js
Modular Design	مشروع غير منظم
Factory و Facade	كود متكرر
سهولة التوسع	صعوبة إضافة صفحات جديدة
ترابط منخفض	ترابط مرتفع

أثر تطبيق الأنماط على المشروع

بعد تطبيق أنماط التصميم أصبح المشروع أكثر تنظيمًا، وأصبح من السهل إضافة صفحات أو وظائف جديدة دون التأثير على بقية أجزاء النظام. كما ساهمت هذه الأنماط في تقليل تكرار الكود، وتحسين قابلية الصيانة، ورفع جودة التصميم العام للتطبيق.

الفصل الثالث عشر: المحاضرة الخامسة – إدارة الأخطاء والاستثناءات (Error & Exception Handling)

مقدمة

تُعد إدارة الأخطاء والاستثناءات من أهم الجوانب في تطوير البرمجيات الحديثة، حيث لا يمكن افتراض أن جميع العمليات ستنفذ دائمًا بصورة صحيحة. فقد يُدخل المستخدم بيانات غير صحيحة، أو قد يحدث خطأ أثناء معالجة الطلبات، أو قد يتعذر الوصول إلى البيانات المطلوبة. لذلك يجب أن يكون النظام قادرًا على اكتشاف الأخطاء والتعامل معها بطريقة تمنع توقف البرنامج وتحافظ على تجربة المستخدم.

في مشروع **نظام إدارة درجات الطلاب** تم الاهتمام بإدارة الأخطاء منذ المراحل الأولى للتطوير، وذلك من خلال التحقق من صحة المدخلات، واستخدام آليات معالجة الأخطاء في Node.js وExpress.js، وإرجاع رسائل واضحة للمستخدم بدلاً من ظهور أخطاء تقنية قد تربكه.

يهدف هذا الفصل إلى توضيح كيفية تطبيق مفاهيم إدارة الأخطاء والاستثناءات على المشروع، مع تقديم أمثلة عملية من العمليات الأساسية مثل إضافة الطالب، والبحث، والتعديل.

أولاً: مفهوم الأخطاء (Errors)

تعريف الأخطاء

الأخطاء (Errors) هي مشكلات تؤثر على تشغيل البرنامج أو بيئة التنفيذ، وغالبًا ما تكون ناتجة عن أسباب لا يستطيع البرنامج معالجتها بسهولة، مثل نفاد الذاكرة أو فشل تشغيل الخادم.

في تطبيقات Node.js قد تظهر أخطاء عند:

- فشل تشغيل الخادم.
- فقدان الاتصال بقاعدة البيانات.
- استهلاك الذاكرة بشكل مفرط.
- تحميل ملفات غير موجودة.

هذه الأخطاء تحتاج إلى تصميم جيد للنظام لتقليل احتمالية حدوثها.

مثال من المشروع

إذا حاول الخادم قراءة ملف بيانات غير موجود، فقد يتوقف التطبيق إذا لم تتم معالجة الخطأ.

قبل التحسين

```
const students = loadStudentsFile();
```

بعد التحسين

```
try {
const students = loadStudentsFile();
} catch (error) {
console.error("تعذر تحميل بيانات الطلاب.");
}
```

بهذا الأسلوب يستمر النظام في العمل مع تسجيل الخطأ.

ثانياً: مفهوم الاستثناءات (Exceptions)

تعريف الاستثناءات

الاستثناءات هي أخطاء متوقعة يمكن للبرنامج اكتشافها ومعالجتها أثناء التشغيل دون أن يتوقف النظام بالكامل.

من أمثلة الاستثناءات في مشروعنا:

- إدخال اسم طالب فارغ.
- إدخال درجة خارج النطاق.
- البحث عن طالب غير موجود.
- محاولة تعديل سجل غير موجود.

مثال عملي

إذا بحث المستخدم عن طالب غير موجود

قبل التحسين

```
student.name
```

إذا كانت قيمة **student** غير موجودة سيؤدي ذلك إلى ظهور خطأ وإيقاف العملية

بعد التحسين

```
if(!student){


return res.status(404).json({
```

الطالب غير موجود"":message

});

}

بهذه الطريقة يحصل المستخدم على رسالة واضحة دون انهيار النظام.

ثالثاً: آلية Try / Catch

مفهومها

تُستخدم كتلة **try** لاحتواء الكود المتوقع أن يحدث فيه خطأ، بينما تستخدم **catch** لمعالجة الخطأ في حال وقوعه.

تعد هذه الآلية من أهم وسائل إدارة الأخطاء في JavaScript وNode.js.

التطبيق في المشروع

عند حفظ بيانات طالب جديد قد يحدث خطأ أثناء تنفيذ العملية

try{

addStudent();

}

```
catch(error){

console.log(error);

}
```

التطبيق العملي

في مشروع إدارة درجات الطلاب يمكن استخدام

```
try{

saveStudent(student);

res.status(200).json({

message:"تمت إضافة الطالب بنجاح"

});

}

catch(error){
```

res.status(500).json({

حدث خطأ أثناء حفظ البيانات"":message

});

}

Try/Catch

- منع توقف التطبيق.
- تسجيل الأخطاء.
- تحسين تجربة المستخدم.
- تسهيل اكتشاف أسباب الأعطال.

رابعاً: التحقق من صحة البيانات (Validation)

المفهوم

التحقق من صحة البيانات يعني التأكد من أن البيانات المدخلة صحيحة قبل تنفيذ أي عملية.

يعد التحقق من البيانات أول خط دفاع ضد الأخطاء.

التطبيق في المشروع

قبل إضافة الطالب يتم التحقق من:

اسم الطالب

يجب ألا يكون فارغًا.

رقم القيد

يجب أن يكون موجودًا.

الدرجة

يجب أن تكون رقمًا.

كما يجب أن تكون ضمن النطاق:

0

إلى

100

مثال

```
if(studentName===""){

return res.status(400).json({

message:"اسم الطالب مطلوب"

});

}
```

د

- منع تخزين بيانات خاطئة.
- تقليل الأخطاء.
- حماية قاعدة البيانات.
- تحسين جودة النظام.

خامساً: رسائل الأخطاء (Error Messages)

المفهوم

يجب أن تكون رسائل الخطأ واضحة ومفهومة للمستخدم.

ولا ينبغي أن تحتوي على تفاصيل تقنية معقدة.

قبل التحسين

Error 500

هذه الرسالة لا توضح سبب المشكلة.

بعد التحسين

الرجاء إدخال اسم الطالب.

ة

تساعد المستخدم على:

- معرفة سبب المشكلة.
- تصحيح الخطأ بسرعة.
- عدم الشعور بأن النظام معطل.

سادساً: تسجيل الأخطاء (Logging)

المفهوم

يقصد بتسجيل الأخطاء حفظ معلومات عن الأخطاء التي تحدث داخل النظام حتى يستطيع المطور مراجعتها لاحقًا.

التطبيق في المشروع

console.error(error);

- Winston
- Morgan

لحفظ الأخطاء داخل ملفات مستقلة.

الفوائد

- معرفة أسباب الأعطال.
- تسهيل صيانة النظام.
- متابعة أداء التطبيق.

سابعاً: معالجة أخطاء البحث

من أكثر العمليات عرضة للأخطاء هي عملية البحث.

قد يبحث المستخدم عن طالب غير موجود.

لذلك تم إضافة تحقق قبل عرض البيانات

if(!student){

return res.status(404).json({

message:"لا توجد بيانات مطابقة"

});

}

ثامناً: معالجة أخطاء التعديل

قبل تنفيذ عملية التعديل يجب التأكد من وجود الطالب

if(student==null){

return res.status(404).json({

message:"لا يمكن تعديل طالب غير موجود"

});

}

تاسعاً: معالجة أخطاء الإضافة

قد يحاول المستخدم إدخال نفس رقم القيد مرتين.

لذلك يجب تنفيذ تحقق

if(registrationExists){

```
return res.status(400).json({

رقم القيد مستخدم بالفعل"":message

});

}
```

عاشراً: أفضل الممارسات (Best Practices)

تم اتباع عدد من الممارسات أثناء تطوير المشروع، أهمها:

- التحقق من جميع المدخلات.
- استخدام رسائل واضحة.
- عدم كشف تفاصيل النظام الداخلية للمستخدم.
- تسجيل الأخطاء للمراجعة.
- استخدام أكواد HTTP المناسبة مثل:
  - 200 عند نجاح العملية.
  - 400 عند وجود خطأ في المدخلات.
  - 404 عند عدم العثور على الطالب.
  - 500 عند حدوث خطأ داخلي.

مقارنة قبل وبعد تطبيق إدارة الأخطاء

بعد التطبيق	قبل التطبيق
استمرار النظام في العمل	توقف التطبيق عند الخطأ
رسائل واضحة للمستخدم	رسائل غير مفهومة
التحقق قبل التنفيذ	عدم التحقق من البيانات
تسجيل الأخطاء ومراجعتها	صعوبة معرفة سبب الخطأ

بعد التطبيق	قبل التطبيق
حماية البيانات من الإدخال الخاطئ	إمكانية تخزين بيانات غير صحيحة

أثر تطبيق إدارة الأخطاء على المشروع

أدى تطبيق مفاهيم إدارة الأخطاء والاستثناءات إلى جعل النظام أكثر استقرارًا واعتمادية. فقد أصبح قادرًا على التعامل مع الحالات غير المتوقعة دون التوقف، كما تحسنت تجربة المستخدم بفضل الرسائل الواضحة والتحقق المسبق من البيانات. ومن ناحية التطوير، أصبح اكتشاف الأعطال ومعالجتها أسهل بفضل تسجيل الأخطاء واستخدام أساليب معالجة موحدة.

الفصل الرابع عشر: المحاضرتان السادسة والسابعة

المتغيرات، هياكل البيانات، واتفاقيات التسمية

(Variables, Data Structures & Naming Conventions)

مقدمة

تعتمد جميع التطبيقات البرمجية على البيانات، لذلك فإن اختيار نوع البيانات المناسب، وطريقة تخزينها، وأسلوب تسمية المتغيرات والدوال، يعد من أهم العوامل التي تؤثر في جودة المشروع.

في مشروع **نظام إدارة درجات الطلاب** تم تصميم البيانات بطريقة بسيطة ومنظمة، بحيث يسهل التعامل معها داخل الواجهة الأمامية والخادم، كما تم الالتزام باتفاقيات التسمية الخاصة بلغة JavaScript وبيئة Node.js، مما جعل الكود أكثر وضوحًا وأسهل في التطوير والصيانة.

وسيتم في هذا الفصل توضيح أنواع البيانات المستخدمة، وهياكل البيانات، وآلية تسمية المتغيرات والدوال، وكيفية تنظيمها داخل المشروع.

أولاً: المتغيرات (Variables)

مفهوم المتغيرات

المتغير هو مساحة في الذاكرة تُستخدم لتخزين قيمة يمكن استخدامها أو تعديلها أثناء تشغيل البرنامج.

وتُستخدم المتغيرات في المشروع لتخزين جميع المعلومات المتعلقة بالطلاب مثل الاسم، ورقم القيد، والدرجة، بالإضافة إلى البيانات المؤقتة المستخدمة أثناء تنفيذ العمليات المختلفة.

في JavaScript تم استخدام الكلمات المفتاحية **let** و **const** وفقًا لطبيعة البيانات، حيث تُستخدم **const** للقيم الثابتة، بينما تُستخدم **let** للقيم التي يمكن تغييرها أثناء التنفيذ.

أمثلة من المشروع

اسم الطالب

let studentName;

يستخدم هذا المتغير لتخزين اسم الطالب الذي يقوم المستخدم بإدخاله.

رقم القيد

let registrationNumber;

يستخدم لحفظ رقم القيد الجامعي.

الدرجة

let studentDegree;

يخزن درجة الطالب بعد إدخالها.

قائمة الطلاب

let students = [];

تمثل جميع الطلاب الموجودين داخل النظام.

لماذا استخدمنا أسماء واضحة؟

بدلاً من استخدام أسماء مثل

let x;

let y;

let z;

تم استخدام

let studentName;

let studentDegree;

;let registrationNumber

لأن الاسم الواضح يختصر الوقت اللازم لفهم الكود ويقلل الحاجة إلى التعليقات.

ثانياً: أنواع البيانات (Data Types)

يعتمد المشروع على عدة أنواع من البيانات، لكل منها وظيفة محددة.

1. String

يستخدم لتخزين النصوص.

مثل:

- اسم الطالب.
- رقم القيد

let studentName = "Ahmed";

2. Number

يستخدم لتخزين القيم الرقمية.

مثل:

- الدرجة.
- الرقم التعريف

```
let degree = 95;
```

3. Boolean

يمثل القيم المنطقية.

يستخدم في المشروع أثناء التحقق من البيانات

```
let isValid = true;
```

4. Object

يمثل الطالب داخل النظا

```
const student = {

id:1,

name:"Ahmed",

regNumber:"2025001",
```

degree:90

;{

يعتبر هذا الكائن الوحدة الأساسية التي يتعامل معها المشروع.

---

5. **Array**

تم استخدام المصفوفة لتخزين جميع الطلاب

;[] = const students

JSON

يستخدم JSON في تبادل البيانات بين الواجهة الأمامية والخادم

{

"name":"Ahmed",

"degree":90

}ويعد JSON من أهم التنسيقات المستخدمة في تطبيقات الويب الحديثة.

ثالثاً: هياكل البيانات (Data Structures)

مفهوم هياكل البيانات

هي الطريقة التي تُنظم بها البيانات داخل البرنامج لتسهيل عمليات البحث، والإضافة، والتعديل، والحذف.

وقد تم اختيار هياكل بيانات بسيطة تتناسب مع حجم المشروع الحالي، مع إمكانية استبدالها مستقبلًا بقاعدة بيانات مثل MongoDB أو MySQL.

(Objectالكائن)

يمثل كل طالب داخل النظام.

ويحتوي على:

- id
- name
- regNumber
- degree

(Arrayالمصفوفة)

تجمع جميع الطلاب

students = [

student1,

student2,

student3

];

؟

لأنها تسمح بـ:

- إضافة طالب.
- حذف طالب.
- تعديل طالب.
- البحث داخل القائمة.

بسهولة.

رابعاً: اتفاقيات التسمية (Naming Conventions)

مفهومها

تعني الالتزام بطريقة موحدة في تسمية:

- المتغيرات.
- الدوال.
- الملفات.
- المجلدات.

وذلك حتى يكون المشروع أكثر احترافية.

Camel Case

تم استخدام Camel Case في تسمية الدوال والمتغيرات

addStudent();

searchStudent();

updateStudent();

;()loadStudents

Pascal Case

يستخدم عند تسمية الكائنات أو الفئات (Classes)

class Student

أسماء الملفات

تم اختيار أسماء الملفات بحيث تعبر عن محتواها.

مثلاً:

بدلاً من

page1.html

خامساً: تنظيم المتغيرات داخل المشروع

تم ترتيب المتغيرات حسب استخدامها

const express = require("express");

const router = express.Router();

;[] = let students

ثم تأتي الدوال بعد ذلك.

وهذا يجعل الملف أكثر تنظيمًا.

سادساً: تنظيم الدوال

بدلاً من كتابة جميع العمليات داخل دالة واحدة

processStudent();

تم تقسيمها إلى

validateStudent();

addStudent();

searchStudent();

updateStudent();

loadStudents();

كل دالة مسؤولة عن عملية واحدة فقط.

وهذا يتوافق مع مبدأ:

Single Responsibility Principle (SRP)

سابعاً: تنظيم الملفات

أصبح المشروع منظمًا بالشكل التالي

Project

```
                                                    |
                                                    ├──── index.js
                                                    |
                                                    ├──── router
                                                    |     └──── studentRoute.js
                                                    |
                                                    ├──── public
                                                    |     ├──── index.html
                                                    |     ├──── add.html
                                                    |     ├──── students.html
                                                    |     ├──── edit.html
```

```
|       ├── reports.html
|       ├── style.css
|       └── script.js
|
└── package.json
```

كل ملف له مسؤولية محددة، مما يسهل فهم المشروع وإدارته.

ثامناً: أثر تطبيق اتفاقيات التسمية

بعد الالتزام باتفاقيات التسمية أصبح المشروع:

- أكثر وضوحًا.
- أسهل في القراءة.
- أقل عرضة للأخطاء.
- أسرع في التطوير.
- مناسبًا للعمل الجماعي.

كما أصبح بإمكان أي مطور جديد فهم المشروع خلال وقت قصير دون الحاجة إلى شرح مفصل.

مقارنة قبل وبعد

بعد التطبيق	قبل التطبيق
أسماء معبرة ودقيقة	أسماء عامة وغير واضحة
ترتيب منطقي للمتغيرات	متغيرات غير منظمة

قبل التطبيق	بعد التطبيق
دوال طويلة	تقسيم الدوال حسب الوظيفة
ملفات غير مرتبة	هيكل مشروع منظم
تكرار في البيانات	استخدام Objects وArrays بطريقة منظمة

العلاقة بين هياكل البيانات وجودة المشروع

اختيار هياكل البيانات المناسبة ساهم في:

- تسريع عمليات البحث.
- تسهيل إضافة الطلاب.
- تبسيط تعديل البيانات.
- تقليل تعقيد الكود.
- تحسين أداء المشروع.

كما أن تنظيم البيانات داخل Objects وArrays يجعل من السهل استبدال التخزين الحالي بقاعدة بيانات مستقبلًا دون الحاجة إلى تغيير منطق التطبيق بالكامل.

ر

المحاضرتان الثامنة والتاسعة

أساسيات بناء البرمجيات المتقدمة (Software Construction Fundamentals)

مقدمة

بعد الانتهاء من تطبيق المبادئ الأساسية لبناء البرمجيات، وكتابة الكود النظيف، واستخدام أنماط التصميم، تأتي المرحلة الأخيرة التي تهدف إلى تقييم جودة المشروع من الناحية الهندسية.

لا تعتمد جودة المشروع على عمله بصورة صحيحة فقط، بل تعتمد أيضًا على سهولة تطويره وصيانته، وإمكانية إعادة استخدام مكوناته، ومدى الترابط بين الملفات، وإمكانية التوسع مستقبلًا.

ولهذا ركزت المحاضرتان الثامنة والتاسعة على مجموعة من المفاهيم المهمة، وهي:

- Coupling
- Cohesion

- Reusability
- Maintainability
- Scalability
- Refactoring
- Software Quality

وقد تم تطبيق هذه المفاهيم بصورة عملية على مشروع **نظام إدارة درجات الطلاب**.

أولاً: التماسك (Cohesion)

مفهوم التماسك

يقصد بالتماسك مدى ارتباط الوظائف الموجودة داخل الملف الواحد ببعضها البعض.

كلما كانت جميع الوظائف داخل الملف تؤدي غرضًا واحدًا فقط كان التماسك مرتفعًا.

ويعد التماسك المرتفع من أهم خصائص المشاريع الاحترافية.

تطبيقه في المشروع

studentRoute.js

. يحتوي فقط على العمليات الخاصة بالطلاب.

مثل:

- addStudent()
- searchStudent()
- updateStudent()
- loadStudents()

ولا يحتوي على:

- تنسيقCSS
- صفحاتHTML
- تشغيل الخادم

وبذلك يحقق الملف:

High Cohesion

مرتفعًا؟ Cohesionلماذا يعتبر

لأن كل ملف يؤدي مهمة واحدة فقط.

ولا توجد ملفات تؤدي أكثر من مسؤولية.

High Cohesionفوائد

- سهولة فهم المشروع.
- سهولة الصيانة.
- سهولة إضافة خصائص جديدة.
- تقليل الأخطاء.
- تسهيل اختبار النظام.

Couplingثانياً: الاقتران (

مفهوم الاقتران

يقصد بالاقتران مقدار اعتماد ملف على ملف آخر.

كلما كان الاعتماد قليلاً كان المشروع أفضل.

ولهذا تسعى جميع المشاريع الحديثة إلى تحقيق:

Low Coupling

قبل تطوير المشروع

كانت جميع العمليات موجودة داخل

index.js

مما يعني أن أي تعديل داخل الملف قد يؤدي إلى حدوث أخطاء في بقية أجزاء المشروع.

بعد التطوير

تم فصل المشروع إلى

index.js

studentRoute.js

HTML

CSS

JavaScript

وأصبح كل ملف مستقلاً عن الآخر.

مثال

صفحة HTML لا تعرف كيفية إضافة الطالب.

كل ما تقوم به هو إرسال الطلب.

أما التنفيذ الحقيقي فيوجد داخل

studentRoute.js

الفوائد

- سهولة التطوير.

- تقليل الأخطاء.
- إمكانية تعديل ملف دون التأثير على بقية الملفات.
- سهولة اختبار النظام.
- **مقارنة بين Coupling و Cohesion**

Low Coupling	• High Cohesion
الملفات مستقلة عن بعضها	كل ملف مسؤول عن وظيفة واحدة
سهولة التعديل	سهولة الصيانة
تقليل الأخطاء	وضوح المشروع
سهولة التطوير	تنظيم الكود

ثالثاً: إعادة استخدام الكود (Reusability)

المفهوم

إعادة استخدام الكود تعني كتابة الوظائف مرة واحدة ثم استخدامها في أكثر من مكان داخل المشروع.

ويعتبر هذا المبدأ من أهم مبادئ بناء البرمجيات لأنه يقلل حجم المشروع ويزيد من جودة الكود.

التطبيق في المشروع

تم إنشاء دوال مشتركة مثل

validateStudent()

loadStudents()

searchStudent()

()updateStudent

واستخدمت هذه الدوال في أكثر من صفحة.

مثال

بدلاً من كتابة كود التحقق داخل:

- add.html

ثم كتابته مرة أخرى داخل:

- edit.html
- تم إنشاء

()validateStudent

ثم استدعاؤها عند الحاجة.

الفوائد

- تقليل حجم المشروع.
- سهولة التطوير.
- سهولة الصيانة.
- تقليل الأخطاء.

Maintainability) رابعاً: قابلية الصيانة

المفهوم

تعني قدرة المطور على تعديل المشروع أو إصلاح الأخطاء أو إضافة خصائص جديدة بسهولة.

كلما كان المشروع منظمًا أصبحت صيانته أسهل.

التطبيق

إذا أراد المطور إضافة خاصية:

"حذف طالب"

فلن يحتاج إلى تعديل جميع الملفات.

يكفي أن يضيف

()deleteStudent

داخل

studentRoute.js

.

وبذلك لا يتأثر باقي المشروع.

لماذا المشروع سهل الصيانة؟

لأن:

- الملفات منفصلة.
- أسماء الدوال واضحة.
- الكود منظم.
- لا يوجد تكرار.

خامساً: قابلية التوسع (Scalability)

المفهوم

يقصد بها إمكانية إضافة وظائف جديدة مستقبلاً دون إعادة كتابة المشروع.

التطبيق

يمكن مستقبلاً إضافة:

- حذف طالب.
- تسجيل دخول.
- إدارة المواد.
- إدارة الأساتذة.
- حساب المعدل.
- رفع ملفات PDF.
- تصدير Excel.
- MongoDB.
- MySQL.

كل ذلك دون إعادة تصميم المشروع.

لماذا؟

لأن المشروع يعتمد على:

Modularity

Router Pattern

Clean Code

سادساً: إعادة هيكلة الكود (Refactoring)

مفهوم Refactoring

هو تحسين الكود دون تغيير وظيفته.

أي أن البرنامج يعطي نفس النتيجة ولكن بكود أفضل.

قبل التطوير

كانت جميع العمليات داخل

index.js

بعد التطوير

تم إنشاء

/router

studentRoute.js

وأصبحت العمليات منفصلة.

الفوائد

- تحسين القراءة.
- تحسين الأداء.
- تحسين الصيانة.
- تحسين جودة المشروع.

سابعاً: جودة البرمجيات (Software Quality)

تم تقييم المشروع بناءً على مجموعة من المعايير.

الوضوح

تم استخدام أسماء واضحة.

التنظيم

تم تقسيم المشروع إلى ملفات مستقلة.

الأداء

العمليات تنفذ بسرعة

سهولة الاستخدام

واجهة بسيطة.

سهولة التطوير

إضافة خصائص جديدة بسهولة.

قابلية الصيانة

يمكن إصلاح الأخطاء بسهولة.

إعادة الاستخدام

الدوال تستخدم في أكثر من مكان.

التوسع

يمكن تطوير المشروع مستقبلاً.

ثامناً: مقارنة بين النسخة الأولى والنسخة النهائية :

النسخة النهائية	النسخة الأولى
عدة صفحات مترابطة	صفحة واحدة
إضافة، تعديل، بحث، عرض، تقارير	إضافة طالب فقط
مشروع مقسم إلى Modules	ملف JavaScript واحد
أسماء واضحة	أسماء عامة

النسخة النهائية	النسخة الأولى
Router مستقل	Router لا يوجد
Validation كامل	Validation لا يوجد
واجهة احترافية	واجهة بسيطة
Reusable Code	كود متكرر
سهولة الصيانة	صعوبة الصيانة
High Quality	Low Quality

تاسعاً: تقييم المشروع النهائي

بعد تطبيق جميع مفاهيم مادة بناء البرمجيات أصبح المشروع يحقق معظم معايير الجودة البرمجية، ويمكن تلخيص ذلك كما يلي

الحالة	المعيار
✅ مطبق	Clean Code
✅ مطبق	DRY
✅ مطبق	Abstraction
✅ مطبق	Modularity
✅ مطبق	Encapsulation
✅ مطبق	Information Hiding
✅ مطبق	Router Pattern
✅ مطبق	Reusability
✅ مطبق	High Cohesion
✅ مطبق	Low Coupling
✅ مطبق	Validation

المعيار	الحالة
Error Handling	✅ مطبق
Naming Convention	✅ مطبق
Maintainability	✅ مطبق
Scalability	✅ مطبق
Refactoring	✅ مطبق

الخاتمة

ُظهر هذا المشروع كيف يمكن تحويل المفاهيم النظرية لمادة **بناء البرمجيات** إلى تطبيق عملي متكامل باستخدام **Node.js** و**Express.js**. فقد بدأ المشروع كنظام بسيط لإضافة بيانات الطلاب، ثم تطور تدريجيًا عبر تطبيق مبادئ **التجريد، والتغليف، والوحدات النمطية، والكود النظيف، وأنماط التصميم، وإدارة الأخطاء، وإعادة الهيكلة** حتى أصبح نظامًا أكثر تنظيمًا واحترافية.

كما ساهم تقسيم المشروع إلى وحدات مستقلة، واستخدام أسماء واضحة، وإعادة استخدام الكود، وتقليل الترابط بين الملفات، في تحسين جودة النظام وجعله أكثر قابلية للصيانة والتطوير. ويُعد هذا المشروع نموذجًا عمليًا يوضح أثر تطبيق مبادئ بناء البرمجيات في إنتاج تطبيقات أكثر موثوقية واستدامة، ويوفر أساسًا مناسبًا لإضافة مزايا مستقبلية مثل إدارة المستخدمين، وربط قاعدة بيانات احترافية، وإصدار التقارير المتقدمة.